AML Assignment 1

Jina You (jy2656)
February 28, 2026

Link to Colab: https://drive.google.com/file/d/1tirialMk4hFB-2mZPyzOneYKqtG7rr4f/view?usp=sharing

1. Problem and Data Description

I used the heart disease data from UCI ML repo. The features (X) dataset has 13 variables, 8 of which are discrete values (categorical features) and the rest are continuous (numerical features)

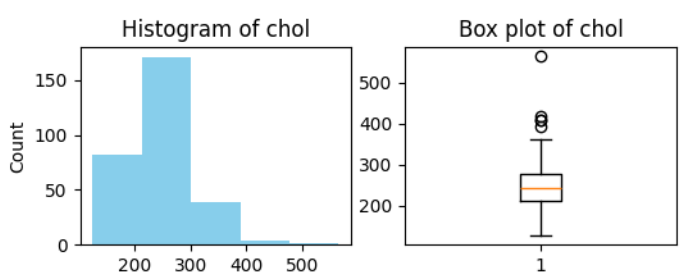
The y values are from {0,1,2,3,4}. Values 1, 2, 3, and 4 all correspond to presence of heart disease, and value 0 corresponds to its absence.

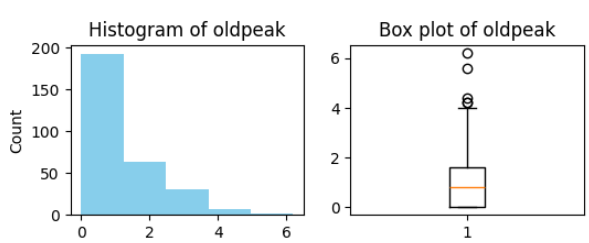
2. Data Cleaning

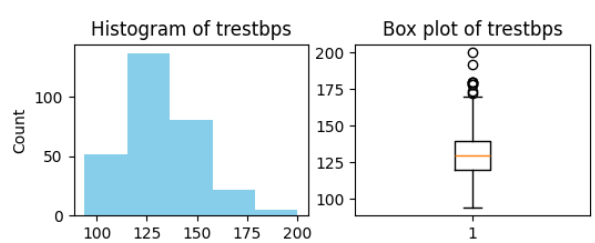
I began the analysis by understanding the data and variables.

For categorical features, I first looked at the unique values and if anything stood out. There were two categorial variables with NaN values, but otherwise the numbers were consistent with the data description.

For numerical values, I looked at the distribution of values using histograms. Some variables, namely chol, trestbps, and oldpeak showed some extreme values. Chol had one value that was much higher than others. Based on literature, chol above 200 is a cause for disease but this person, with chol of over 574, had no disease. I considered this data an outlier and removed the row from the clean dataset. For the other two (trestbps and oldpeak), the extreme values were relatively closer to the rest of the data, and when I tried to remove them from the dataset, the model performed more poorly, so I left them.

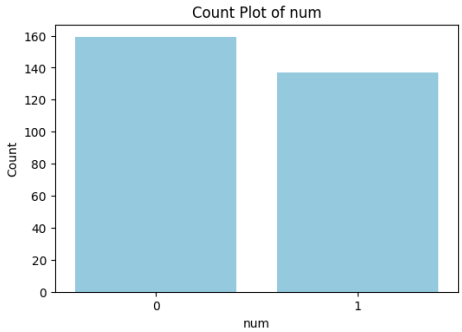






Then, I looked for rows with NaN values in any column (ca and thal, in particular). Since there were only 6 rows out of 303 (< 2%), I dropped them from the clean dataset as I thought it was a small enough proportion that removing those rows wouldn't affect the model performance too much.

For the y data, I applied the threshold of 0.5 to classify anything greater than or equal to 1 (e.g., 1, 2, 3, 4) as "1", which means that heart disease exists. The distribution of presence vs absence of heart disease seems even.



3. Feature engineering

Basics:

I one hot encoded the categorical variables. There were some categorical variables that seemed to be ordered, as in higher numbers actually meant higher values of something. These were 1. Ca, 2. Slope, 2. Fbs. I initially considered excluding them from the one hot encoding process and consider them numerical variables instead. The results seemed to be better in some cases, but not so much in others. In the final results I show below, this choice made no difference so I considered all 8 features to be categorical values. Later, after feature engineering 'age' variable as well, I considered 'age' to be categorical variable as well.

I standardized the numerical values. The choice of minmax scaling and standardizing didn't seem to make much difference in the model results. In theory, I would minmax scale variables which seem to be bounded or variables without much extreme values. Although minmax scaling could have been a better choice for age variable, there were some extreme variables in some other features that I didn't want to shrink using min max scaling, so I chose standard scaling for all numerical features. Also, scaling is more meaningful for linear regression models because the actual magnitude of the values matter, whereas Naïve Bayes relies on probability of classes. I still applied scaling for both types of models.

Further feature engineering:

I bucketed age to four groups: below 40, between 40 and 55, between 55 and 70, and above 70. I made this choice because those below 40 are probably less likely to have a heart disease than those older, so I made narrower bins for those between ages 40 and above.

Based on some online search, serum cholesterol (chol) of below 200 is a healthy range, and 200-240 is borderline high, and above 240 is high. I bucketed the values accordingly.

Finally, I created a new ratio variable thalach_trest:= thalach/trestbps. I did this because I wanted to use the maximum heart rate as a ratio of resting blood pressure as a variable. The idea was that those with high resting blood pressure would also reach relatively higher heartrate. After the feature engineering, seems like the importance of thalach and trestbps variables increased.

Before:

exang	0.167606
sex	0.138892
ca	0.123733
fbs	0.098097
cp	0.087593
slope	0.062678
thal	0.060929
chol	0.049789
restecg	0.038070
age	0.034748
oldpeak	0.019516
trestbps	0.002185
thalach	0.001900

After:

thalach_trest	0.246041
exang	0.167223
sex	0.136353
ca	0.122928
fbs	0.095406
cp	0.087368
slope	0.062889
thal	0.061936
chol	0.050424
restecg	0.038373
age	0.035967
oldpeak	0.020333

4. Model Training

I used 25% as test sample size and used the same sample for training and testing of all models for fair comparison. I made use of stratified sampling.

For the Gaussian Naïve Bayes and linear regression, I didn't apply smoothing for the results shown because the class distribution was fairly even. Still, I tested different smoothing variables and it seemed that the smoothing coefficient of 0.1 or 1 showed better results than using smaller coefficients.

One of the distinctions (in implementation) between the Gaussian Naïve Bayes and linear regressions is that the results of linear regressions had to be converted to 0 and 1 using a threshold value, while the naïve bayes model is probability based so the threshold selection is not necessary. In the next section, I have added a graph that shows the model performance for different threshold values.

5. Evaluation

As a first step, I ran the models using minimal data work and without feature engineering other than one hot encoding and standard scaling (only the "basics" above). I ran Gaussian Naïve Bayes (because the numerical variables are continuous) as well as Linear Regression (without regularization).

Results:

Gaussian NB (train): Accuracy=0.8378 | Precision=0.8602 | Recall=0.7767 | F1-score=0.8163

Gaussian NB (test): Accuracy=0.8919 | Precision=0.8824 | Recall=0.8824 | F1-score=0.8824

```
Linear Regression (train): Accuracy=0.8784 | Precision=0.9130 | Recall=0.8155 | F1-score=0.8615
Linear Regression (test): Accuracy=0.8108 | Precision=0.7941 | Recall=0.7941 | F1-score=0.7941
```

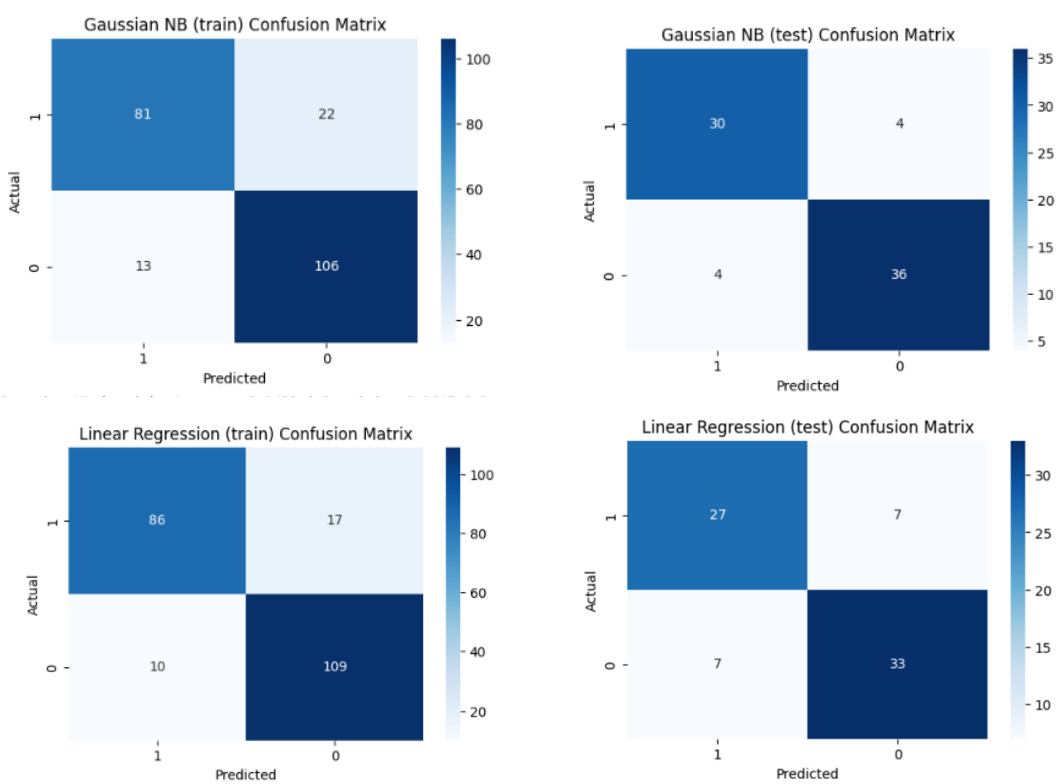
- Naïve Bayes accuracy was 0.84 for the train set and 0.88 for test set. F1 score was 0.82 for train set and 0.88 for test set. Results show signs of good generalization.
- Linear regression accuracy was 0.88 for train set and 0.81 for test set. F1 score was 0.86 for train set and 0.79 for test set. There is a slight sign of overfit.

I rely on accuracy because the classes are balanced it is calculated using all four components of classification matrix (true positive, true negative, false positive, false negative). That being said, since this model is used to detect heart disease, and it is dangerous if a heart disease goes undetected, an important metric to look at is the rate of false negatives, i.e., recall. Although striking a good balance between precision and recall is important (F1 is still important), I would lean towards getting a higher recall at the expense of a bit lower precision. This point is discussed more towards the end of the report.

Next, I show the results after further feature engineering. There was no improvement in the test set. There was still sign of overfitting in the linear regression model.

Results:

```
Gaussian NB (train): Accuracy=0.8423 | Precision=0.8617 | Recall=0.7864 | F1-score=0.8223
Gaussian NB (test): Accuracy=0.8919 | Precision=0.8824 | Recall=0.8824 | F1-score=0.8824
Linear Regression (train): Accuracy=0.8784 | Precision=0.8958 | Recall=0.8350 | F1-score=0.8643
Linear Regression (test): Accuracy=0.8108 | Precision=0.7941 | Recall=0.7941 | F1-score=0.7941
```



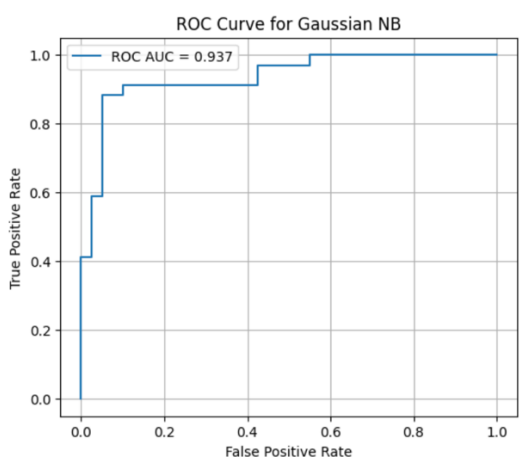
- Gaussian Naïve Bayes showed test set accuracy of 0.89 and F1 score of 0.88. No improvement compared to the first set of results.
- Linear regression showed test set accuracy of 0.81 vs train set accuracy of 0.88. F1 score for test set was 0.79 vs 0.86 for train set. There is a sign of overfitting.

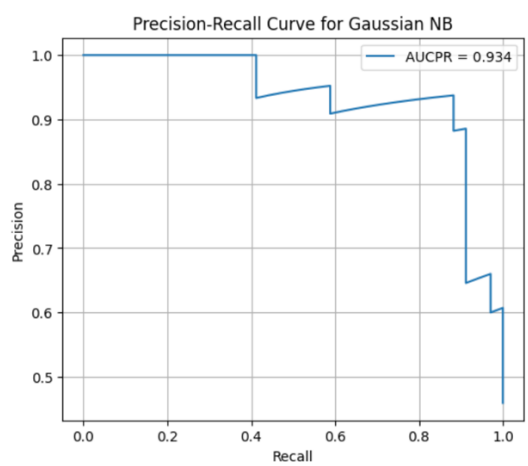
To improve the issue of overfitting, I applied L1 an L2 regularizations. I applied Laplace smoothing with alphas of 0.01, 0.05, 0.1, 0.2, 0.5, and 1 to both lasso and ridge regressions. Using alphas of 0.2 and 0.5 caused the models to predict no positives (non-existence of heart disease). Alpha of 0.05 seemed to generate the best results. The results below are after applying Laplace smoothing with alpha 0.05. Though ridge regression didn't change the results, lasso regression helped improve the generalizability of the linear regression model, as shown by the improved performance in the test set in comparison to the training set. Although unfortunately, recall has not improved, since precision turned out to be high, this can be improved by adjusting the threshold.

alpha 0.05

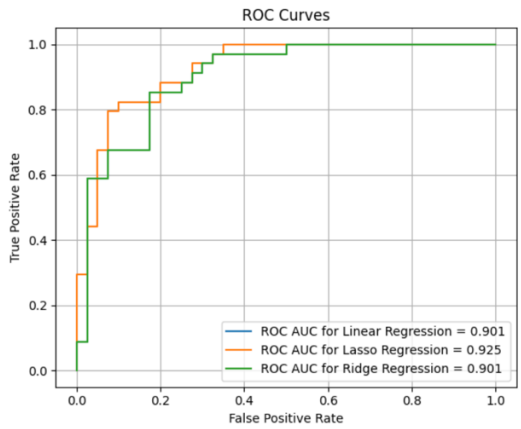
```
Lasso Regression (train): Accuracy=0.8423 | Precision=0.8953 | Recall=0.7476 | F1-score=0.8148
Lasso Regression (test): Accuracy=0.8649 | Precision=0.9000 | Recall=0.7941 | F1-score=0.8438
Ridge Regression (train): Accuracy=0.8784 | Precision=0.8958 | Recall=0.8350 | F1-score=0.8643
Ridge Regression (test): Accuracy=0.8108 | Precision=0.7941 | Recall=0.7941 | F1-score=0.7941
```

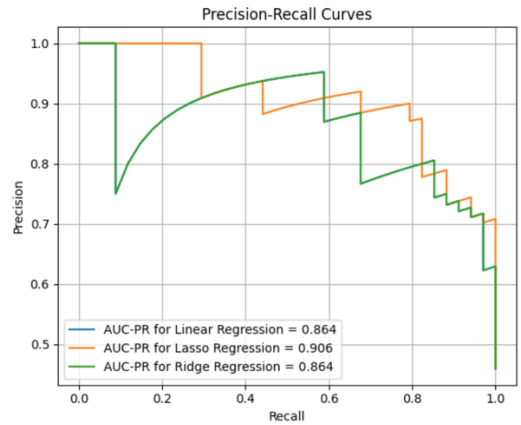
Since Naïve Bayes had the best results, I plot the ROC and Precision-recall curves for that model. The ROC AUC is 0.937 and AUCPR is 0.934 which seem pretty good.





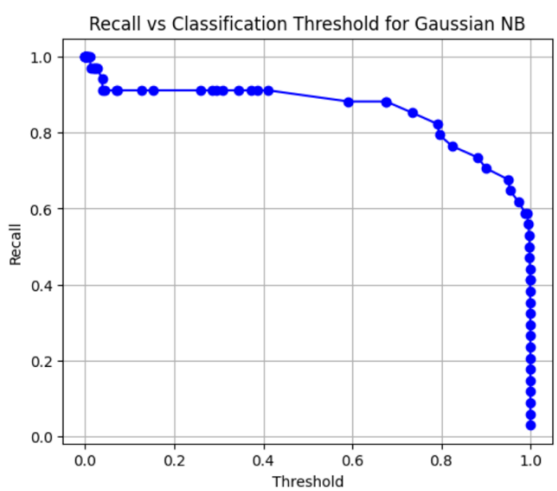
Out of the regression models, Lasso regression showed the highest AUC overall.





It was surprising to see better results from lasso regression, because I thought all of the features had some sort of importance in predicting heart disease (as opposed to select subset). Lasso regression would be more ideal for promoting sparsity whereas ridge regression would incorporate all features with scaled down coefficients.

As stated above, an important metric to look at is the rate of false negatives. Although striking a good balance between precision and recall is important, I would lean towards getting a higher recall at the expense of a bit lower precision. Based on the Precision-Recall Curve for Gaussian NB, somewhere around a 0.9 recall could be a sweet spot. That would give us 0.85~0.90 precision. This can be achieved by adjusting the threshold. See chart below; a threshold of 0.4 would help us achieve 0.9 recall. Unfortunately, further decrease in the threshold barely improves recall.



I have tested different threshold values for the regression models as well. See code outputs.

6. AI Tool Usage Disclosure

I used ChatGPT. After I built the code structure using class notes and shared colab workbook, I used it for syntax and debugging. I also used it for a lot of the codes in the evaluations section. For example, I got AI help with outputting different evaluation metrics such as accuracy, recall,

threshold, precision, and f1. In general, I knew what I wanted to see (such as the threshold vs. recall curve and the correlation heatmap) but didn't know how to write the code, so I asked AI.

I also used it to improve model implementation. For example, when I was trying out validation (which I didn't include in the final submission), I wanted to be able to apply the same kind of preprocessing to the validation step. I also wanted to separate the processing of numerical and categorical values. I learned from AI how to use things like Pipeline and ColumnTransformer.